



INTRODUÇÃO AO MATLAB

1. Introdução e Objetivos

Esta ficha de trabalho tem como finalidade a introdução prática ao ambiente MATLAB, privilegiando a passagem de uma lógica de programação imperativa tradicional, assente no uso de ciclos, para uma abordagem vetorizada, recorrendo aos Live Scripts.

No final da realização desta ficha, o aluno deverá ser capaz de:

- Manipular matrizes e vetores de forma eficiente;
- Aplicar *Logical Indexing* na seleção e filtragem de dados;
- Produzir visualizações de dados claras e corretamente formatadas;
- Desenvolver funções modulares, devidamente estruturadas e documentadas.

2. Recursos e Bibliografia

- MathWorks. (s.d.). MATLAB documentation. MathWorks Help Center. <https://www.mathworks.com/help/matlab/>
- MathWorks. (s.d.). MATLAB Onramp. <https://www.mathworks.com/learn/tutorials/matlab-onramp.html>
- MathWorks. (s.d.). Live Scripts. MathWorks Help Center. <https://www.mathworks.com/help/matlab/live-scripts.html>
- Material de apoio da disciplina. (s.d.). Sebenta e recursos disponibilizados na plataforma Moodle.

3. Tarefas Práticas

As tarefas propostas nesta secção têm como objetivo consolidar, através da prática, os principais conceitos abordados anteriormente. Pretende-se que o aluno desenvolva competências na manipulação eficiente de dados, na visualização gráfica e na implementação de soluções estruturadas em MATLAB, privilegiando abordagens vetorizadas e boas práticas de programação.

Ao longo das atividades, será incentivada a análise crítica dos resultados obtidos, bem como a comparação entre diferentes estratégias de resolução, promovendo uma compreensão mais profunda das potencialidades do ambiente MATLAB em contexto de Engenharia e Informática.

3.1. Manipulação de Matrizes e Vetores (Abordagem Vectorizada)

Escreve os comandos MATLAB que permitam executar as seguintes tarefas, privilegiando a eficiência e a utilização de operações vetorizadas:

a) Geração de sequências

Cria um vetor linha **v_pares** que contenha todos os números pares entre 2 e 100.

b) Transformação

Obtém o vetor coluna correspondente (transposta) do vetor **v_pares**.

c) Matrizes identidade e constantes

Cria uma matriz **M_ones** de dimensão 3×3 , preenchida com o valor 1.

De seguida, altera o elemento da primeira linha e terceira coluna para o valor 33.

d) Distribuições aleatórias

- Gera uma matriz A de dimensão 8×8 com valores provenientes de uma distribuição normal padrão (**randn**);
- Gera uma matriz B de dimensão 8×8 com valores uniformemente distribuídos no intervalo $[0, 1]$ (**rand**).

Reflexão:

- Compara visualmente os histogramas das matrizes A e B.
- Que diferenças observas nas distribuições dos valores?

(Dica: utiliza o comando **histogram(A)**.)

d) Redimensionamento

Remove a segunda coluna da matriz A.

3.2. Visualização de Dados e Estética

A visualização de dados desempenha um papel fundamental em Engenharia, permitindo interpretar e comunicar resultados de forma eficaz. Nesta tarefa, pretende-se criar um gráfico comparativo entre duas funções trigonométricas.

a) Domínio

Define um domínio x no intervalo $[0, 2\pi]$, com um passo de $\pi/100$.

b) Representação gráfica

Cria um gráfico que contenha as funções $\sin(x)$ e $\cos(x)$.

c) Apresentação e formatação

Em vez de recorrer apenas ao comando **plot**, utiliza as seguintes opções para tornar o gráfico mais informativo e profissional:

- **Legenda:** identifica claramente cada função representada;
- **Anotações:** adiciona um título ao gráfico e rótulos aos eixos (**xlabel**, **ylabel**);
- **Estilo:** representa a função seno com uma linha azul tracejada e a função cosseno com uma linha verde contínua, com maior espessura (**Linewidth**);
- **Grelha:** ativa a grelha (**grid on**) para facilitar a leitura do gráfico.

3.3. Programação e Lógica (M-files vs. Funções)

Nesta secção, pretende-se comparar a programação tradicional, baseada em ciclos (**for**, **if**), com abordagens mais eficientes do MATLAB, nomeadamente a utilização de indexação lógica.

a). Geração e Filtragem de Dados

Cria uma função **processa_dados(n, m, limite)** que:

1. Gere uma matriz **R** de dimensões $n \times m$ com números inteiros aleatórios entre 1 e **limite** (utiliza o comando **randi**);
2. Retorne um vetor contendo todos os elementos pares dessa matriz.

Desafio:

Implementa esta funcionalidade de duas formas distintas:

- **Versão A:** utilizando ciclos **for** e uma condição **if**;
- **Versão B:** utilizando apenas uma linha de código com *Logical Indexing* (ex.: **R(mod(R,2)==0)**).

b). Interatividade e Robustez

Cria um *script* ou função **analise_estatistica** que:

1. Solicite ao utilizador a introdução sucessiva de valores numéricos (utiliza **input**);
2. Termine a leitura quando o utilizador introduzir o valor 0;
3. No final, apresente:
 - o maior valor introduzido;
 - a média dos valores (excluindo o zero);
 - o número de valores positivos introduzidos.

Nota:

Procura implementar uma solução que armazene os dados num vetor dinâmico e recorra às funções **max**, **mean** e **sum**.

c) Processamento de “Sinais” Simples

Cria uma função **soma_condicional(matriz)** que receba uma matriz e devolva a soma de todos os elementos que sejam simultaneamente ímpares e positivos.

Compara o tempo de execução (utilizando **tic** e **toc**) entre:

- uma implementação baseada em ciclos;
- uma implementação vetorizada, recorrendo a **find** ou à indexação lógica.

3.4. Processamento de Imagem como Matriz

As imagens digitais podem ser representadas como matrizes, o que permite aplicar diretamente técnicas de processamento numérico.

1. Carrega uma imagem de exemplo do MATLAB:

```
I = imread('pout.tif');
```

2. Visualiza a imagem original:

```
imshow(I);
```

3. **Tarefa:** cria uma nova imagem **I_bin**, correspondente à versão binária da imagem original, em que os píxeis com intensidade superior a um determinado limiar (por exemplo, 128) assumem o valor branco, e os restantes o valor preto.

Dica: **I_bin = I > 128;**

4. Visualiza a imagem resultante e analisa de que forma a manipulação de matrizes se aplica a dados reais.